Data Structures & Algorithms in C

Complete Handbook

Written by

D Charan Jeet

A comprehensive textbook for engineering students

Beginner to Intermediate Level

Contents

Introduction **6**

1 Foundations of Data Structures & Complexity **7**

1.1 Introduction to DSA . . . 7
1.1.1 What is a Data Structure? . . . 7
1.1.2 What is an Algorithm? . . . 8
1.1.3 Why DSA Matters in Interviews & Real Systems . . . 8
1.1.4 Abstract Data Types (ADT) . . . 9
1.2 Time & Space Complexity . . . 9
1.2.1 Big-O Notation . . . 9
1.2.2 Best, Worst, and Average Case . . . 10
1.2.3 Common Complexities . . . 10
1.2.4 Basic Mathematical Analysis . . . 11
1.3 Arrays . . . 12
1.3.1 What is an Array? . . . 12
1.3.2 Static vs. Dynamic Arrays . . . 12
1.3.3 Traversal . . . 13
1.3.4 Insertion & Deletion . . . 13
1.3.5 Prefix Sum . . . 14
1.3.6 Two Pointer Technique (Basic Introduction) . . . 14
1.4 Strings . . . 16
1.4.1 What is a String? . . . 16
1.4.2 String Traversal . . . 16
1.4.3 Palindrome Checking . . . 17
1.4.4 Frequency Counting . . . 17
1.4.5 Basic Pattern Matching . . . 18
1.5 Recursion (Important Foundation) . . . 18
1.5.1 What is Recursion? . . . 19
1.5.2 Base Case & Recursive Case . . . 19
1.5.3 Stack Memory Concept . . . 20
1.5.4 Classic Problem: Factorial . . . 20
1.5.5 Classic Problem: Fibonacci . . . 21
1.5.6 Classic Problem: Sum of Digits . . . 21
1.5.7 Classic Problem: Reverse an Array . . . 22

2 Linear Data Structures **24**

2.1 Linked List . . . 24
2.1.1 What is a Linked List? . . . 24
2.1.2 Node Structure in C . . . 25
2.1.3 Singly Linked List — Insert at Beginning . . . 25
2.1.4 Doubly Linked List . . . 26
2.1.5 Circular Linked List . . . 27
2.1.6 Reverse a Linked List . . . 27
2.1.7 Detect Cycle — Floyd's Algorithm . . . 27
2.2 Stack . . . 28
2.2.1 What is a Stack? . . . 28
2.2.2 Stack Using Array . . . 29
2.2.3 Stack Using Linked List . . . 30
2.2.4 Application: Parenthesis Checking . . . 30
2.2.5 Application: Infix to Postfix . . . 31
2.2.6 Application: Next Greater Element . . . 31
2.3 Queue . . . 31
2.3.1 What is a Queue? . . . 32
2.3.2 Simple Queue Using Array . . . 32
2.3.3 Circular Queue . . . 33
2.3.4 Deque (Double-Ended Queue) . . . 33
2.3.5 Priority Queue (Introduction) . . . 33
2.3.6 Application: Sliding Window Maximum . . . 33
2.4 Hashing . . . 33
2.4.1 What is Hashing? . . . 34
2.4.2 Hash Table Basics . . . 34
2.4.3 Collision Handling . . . 34
2.4.4 Frequency-Based Problems . . . 35

3 Non-Linear Data Structures **37**
3.1 Trees . . . 37
3.1.1 What is a Tree? . . . 37
3.1.2 Binary Tree . . . 38
3.1.3 Tree Traversals . . . 38
3.1.4 Height, Depth, and Leaf Nodes . . . 40
3.1.5 Diameter of Tree . . . 40
3.2 Binary Search Tree (BST) . . . 41
3.2.1 What is a BST? . . . 41
3.2.2 Search in BST . . . 41
3.2.3 Insert in BST . . . 41
3.2.4 Valid BST Check . . . 42

3.2.5 Lowest Common Ancestor (LCA) . . . 42
3.3 Heap . . . 42
3.3.1 What is a Heap? . . . 43
3.3.2 Heap Operations . . . 43
3.3.3 Heap Sort . . . 44
3.3.4 Kth Largest Element . . . 45
3.4 Graphs . . . 45
3.4.1 What is a Graph? . . . 45
3.4.2 Graph Representation . . . 46
3.4.3 BFS (Breadth-First Search) . . . 47
3.4.4 DFS (Depth-First Search) . . . 48
3.4.5 Cycle Detection (Basic) . . . 48
3.4.6 Connected Components . . . 48

4 Algorithms & Problem-Solving Patterns 50
4.1 Searching Algorithms . . . 50
4.1.1 Linear Search . . . 50
4.1.2 Binary Search . . . 50
4.1.3 Binary Search on Answer . . . 51
4.2 Sorting Algorithms . . . 52
4.2.1 Bubble Sort . . . 52
4.2.2 Selection Sort . . . 52
4.2.3 Insertion Sort . . . 53
4.2.4 Merge Sort . . . 53
4.2.5 Quick Sort . . . 54
4.2.6 Comparison of Sorting Techniques . . . 54
4.3 Greedy Algorithms . . . 55
4.3.1 What is a Greedy Algorithm? . . . 55
4.3.2 Activity Selection Problem . . . 55
4.3.3 Coin Change (Greedy Case) . . . 55
4.3.4 Fractional Knapsack . . . 56
4.4 Recursion to Backtracking . . . 57
4.4.1 What is Backtracking? . . . 57
4.4.2 Generate All Subsets . . . 57
4.4.3 Generate All Permutations . . . 57
4.4.4 N-Queens Problem (Conceptual) . . . 58
4.5 Introduction to Dynamic Programming . . . 58
4.5.1 What is Dynamic Programming? . . . 58
4.5.2 Fibonacci with DP . . . 59
4.5.3 Climbing Stairs . . . 59

4.5.4 0/1 Knapsack (Introductory Idea) 60
4.5.5 Longest Common Subsequence (Conceptual) 60
4.6 Sliding Window & Two Pointer Pattern 61
4.6.1 What is the Sliding Window? 61
4.6.2 Fixed Window 62
4.6.3 Variable Window 62
4.6.4 Subarray Problems 63

Introduction

Why DSA Matters

Data Structures and Algorithms (DSA) are the **building blocks of every software system** you use. When you search on Google, order food on an app, or send a message — behind the scenes, data structures organize information and algorithms process it efficiently.

For engineering students, DSA is essential for three reasons:

1. **Technical Interviews:** Companies like Google, Amazon, and Microsoft test DSA skills heavily.
2. **Competitive Programming:** Contest success depends on choosing the right data structure and algorithm.
3. **Real Systems:** Databases, operating systems, networks — all rely on efficient data management.

How to Study This Book

1. **Read the concept first.** Understand *why* something exists before *how* it works.
2. **Trace through the code** by hand on paper (dry run) before running it on a computer.
3. **Write the code yourself.** Do not copy-paste. Type every line.
4. **Solve the practice problems** at the end of each section.
5. **Review the interview tips** — they highlight what interviewers actually look for.

Golden Rule

Think first, code later. The best programmers spend more time thinking about the solution than typing code. Build the logic on paper, then translate it to C.

Chapter 1

Foundations of Data Structures & Complexity

1.1 Introduction to DSA

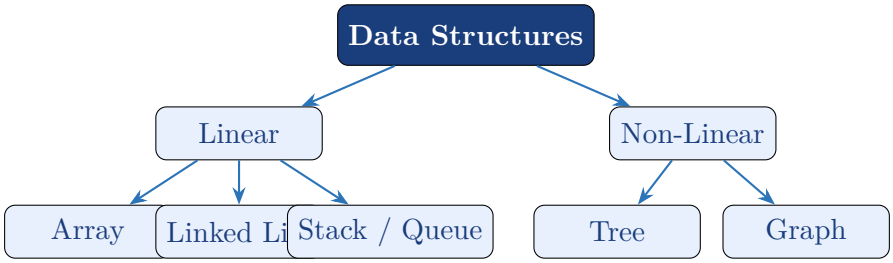
1.1.1 What is a Data Structure?

A **data structure** is a way of organizing and storing data so that we can use it efficiently. Just like you organize your clothes in a wardrobe (shirts in one section, pants in another), data structures organize information in a computer's memory so that it can be accessed and modified quickly.

Real-Life Analogy

Think of a **bookshelf**. You can throw all books in a pile (unorganized) or arrange them by subject and author (organized). The organized bookshelf lets you find any book quickly — that is what a data structure does for data. Different shelf arrangements work better for different needs, just like different data structures work better for different problems.

Data structures are classified as follows:



- **Primitive:** int, float, char — basic types provided by the language.
- **Non-Primitive:** Arrays, Linked Lists, Trees, Graphs — built using primitive types.
- **Linear:** Elements arranged sequentially — each element has a unique predecessor and successor (Array, Stack, Queue, Linked List).
- **Non-Linear:** Elements arranged hierarchically or in networks — one element can connect to multiple others (Tree, Graph).

Why Classification Matters

Choosing the right data structure is half the problem solved. Need fast lookups? Use a hash table. Need sorted data with quick insertion? Use a BST. Need LIFO order? Use a stack. Understanding the classification helps you pick the right tool.

1.1.2 What is an Algorithm?

An **algorithm** is a step-by-step procedure to solve a problem. Think of it as a recipe: clear instructions that, if followed exactly, always produce the correct result.

Real-Life Analogy

Imagine a **recipe for making tea**: (1) Boil water, (2) Add tea leaves, (3) Wait 3 minutes, (4) Strain into cup, (5) Add sugar. These are clear, finite steps with a defined result. That is exactly what an algorithm is — a precise recipe for solving a problem.

Every algorithm must be:

1. **Finite** — it must stop after a certain number of steps. An algorithm that runs forever is broken.
2. **Definite** — each step must be precisely defined with no ambiguity.
3. **Have Input** — zero or more inputs must be provided.
4. **Have Output** — at least one meaningful output must be produced.
5. **Effective** — each step must be simple enough to be carried out in a reasonable time.

1.1.3 Why DSA Matters in Interviews & Real Systems

In interviews, DSA tests your **problem-solving ability**. Companies do not want you to memorize solutions — they want to see how you *think*, how you break down a problem, and how you choose the right approach.

In real systems, the difference is massive:

- **Google Search** uses sophisticated graph algorithms and inverted index data structures to search billions of web pages in milliseconds.
- **GPS navigation** uses shortest-path algorithms (like Dijkstra's) to find the best route.
- **Social media feeds** use priority queues and sorting algorithms to show you the most relevant content.

Choosing the wrong data structure can make your program **1000x slower**. For example,

searching in an unsorted array takes $O(n)$, but searching in a balanced BST takes $O(\log n)$. For 1 million items, that is 1,000,000 vs. 20 operations.

1.1.4 Abstract Data Types (ADT)

An **ADT** defines *what* operations a data structure supports, without specifying *how* they are implemented. It separates the "what" from the "how."

For example, a **Stack ADT** defines three operations:

- `push(x)` — add element x to the top
- `pop()` — remove and return the top element
- `peek()` — look at the top element without removing it

But the *implementation* can be done using an array or a linked list. The user of the stack does not need to know which one is used internally.

Interview Tip

Interviewers often ask: "What is the difference between a data structure and an ADT?" Answer: An ADT is the *interface* (what operations are supported), the data structure is the *implementation* (how those operations work internally). This shows you understand abstraction.

1.2 Time & Space Complexity

1.2.1 Big-O Notation

When we write an algorithm, we need a way to measure how fast it runs. We cannot just time it with a stopwatch because different computers run at different speeds. Instead, we count the **number of operations** the algorithm performs as the input size n grows.

Big-O notation describes the *upper bound* of how fast an algorithm's running time grows. It answers: "In the worst case, how does the time scale as input grows?"

Real-Life Analogy

Imagine you need to find a friend in a crowd of n people. If you check *every person one by one*, that is $O(n)$ — if the crowd doubles, your search time doubles. If the crowd is sorted by height and you use a smarter method (halving each time), that is $O(\log n)$ — if the crowd doubles, you only need *one extra check*.

How to Read Big-O

$O(n)$ means: "The time grows *at most* proportionally to n." It does *not* mean the algorithm takes exactly n steps. It could take $2n$ or $5n + 100$ steps — we drop constants and lower-order terms because Big-O focuses on *growth rate*, not exact count.

Rules:

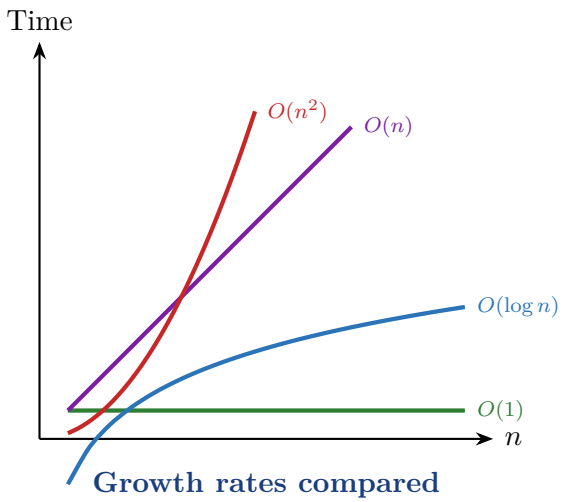
1. Drop constants: $O(3n)$ becomes $O(n)$
2. Drop lower-order terms: $O(n^2 + n)$ becomes $O(n^2)$
3. Multiply nested operations: A loop inside a loop $\rightarrow O(n) \times O(n) = O(n^2)$

1.2.2 Best, Worst, and Average Case

- **Best Case:** The minimum time (e.g., the element you are searching for is at the very first position). We rarely focus on this because it is too optimistic.
- **Worst Case:** The maximum time (e.g., the element is not in the array at all, so you check everything). This is what interviews and Big-O focus on.
- **Average Case:** Expected time over all possible inputs. Useful in practice but harder to calculate.

1.2.3 Common Complexities

Notation	Name	Example
$O(1)$	Constant	Accessing array element by index
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort



Growth rates compared

Common Mistake

Do not confuse $O(n)$ and $O(n^2)$. For $n = 1{,}000{,}000$: $O(n)$ does 1 million operations, $O(n^2)$ does 1 *trillion*. That is the difference between 1 second and 11 days! Always aim for the lowest complexity possible.

1.2.4 Basic Mathematical Analysis

To find time complexity, count the number of basic operations as a function of n. Here is a step-by-step method:

Step 1: Identify the loops and how many times they run.
Step 2: Count the work done inside each loop.
Step 3: Multiply them together for nested loops.

```c
// Example: Sum of array €"â O(n)
int sum = 0;
for (int i = 0; i < n; i++) { // Runs n times
    sum += arr[i]; // O(1) per iteration
}
// Total: n * O(1) = O(n)
```

```c
// Example: Nested loop €"â O(n^2)
for (int i = 0; i < n; i++) { // Runs n times
    for (int j = 0; j < n; j++) { // Runs n times for each i
        printf("%d ", arr[j]); // O(1)
    }
}
// Total: n * n * O(1) = O(n^2)
```

Interview Tip

In interviews, always state the time and space complexity of your solution *before* writing code. This shows you think about efficiency first. If asked "Can you do better?", try to reduce $O(n^2)$ to $O(n \log n)$ or $O(n)$.

Practice Problems:

1. What is the time complexity of a loop that runs from 1 to $n/2$?
2. What is the complexity of a loop where i doubles each time (`i *= 2`)?
3. Analyze the complexity of three nested loops each running n times.

1.3 Arrays

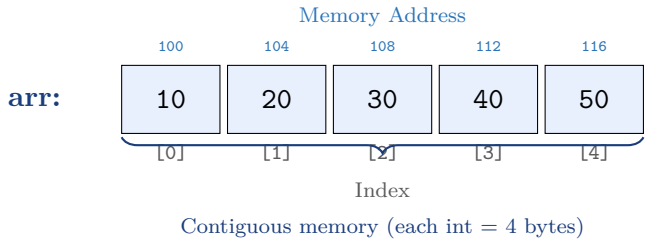
1.3.1 What is an Array?

An **array** is a collection of elements of the same type stored in **contiguous memory locations**. This means all elements sit next to each other in memory, which makes accessing any element by its index extremely fast — $O(1)$.

Real-Life Analogy

Think of a row of **numbered lockers** in a school hallway. Each locker holds one item, and you can directly go to locker #5 without opening lockers 1–4. That direct access is what makes arrays fast. But if you want to insert a new locker in the middle, you have to shift all the lockers after it — that is the downside.

Why do we need arrays? Without arrays, you would need a separate variable for each value: `int a, b, c, d, e;` — this is impractical for 1000 values. Arrays let you store and process collections of data efficiently.



1.3.2 Static vs. Dynamic Arrays

- **Static Array:** Size is fixed at compile time. You must know the size before running the program. Cannot grow or shrink.
- **Dynamic Array:** Size is decided at runtime using `malloc()`. You can allocate exactly the amount of memory you need.

```
// Static array €"â size must be a constant
int arr[5] = {10, 20, 30, 40, 50};

// Dynamic array €"â size can be a variable
int *arr = (int *)malloc(n * sizeof(int));
// IMPORTANT: Don't forget to free when done!
free(arr);
```

Common Mistake

In C, arrays do **not** check boundaries. If you access `arr[10]` in an array of size 5, C will *not* give an error — it will read garbage memory or crash. Always check your indices!

1.3.3 Traversal

Traversal means visiting every element of the array one by one. This is the most basic operation and is the building block for almost every array algorithm.

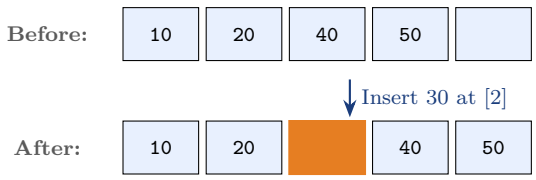
```c
#include <stdio.h>
void traverse(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]); // Access element at index i
    }
    printf("\n");
}
```

Time Complexity: $O(n)$ — you visit each element exactly once.
Space Complexity: $O(1)$ — no extra memory used.

1.3.4 Insertion & Deletion

Insertion and deletion in arrays require **shifting elements**, which makes them slow for large arrays.

Insertion at position `pos`: Shift all elements from `pos` to end one position right, then place the new element.




```c
// Insert 'val' at 'pos' in array of size 'n'
void insert(int arr[], int *n, int pos, int val) {
    for (int i = *n; i > pos; i--) {
        arr[i] = arr[i - 1]; // Shift right
    }
    arr[pos] = val;
    (*n)++;
}
```

Dry Run: Insert 30 at position 2 in `{10, 20, 40, 50}`:

1. $i = 4$: `arr[4] = arr[3]` → `{10, 20, 40, 50, 50}`
2. $i = 3$: `arr[3] = arr[2]` → `{10, 20, 40, 40, 50}`
3. $i = 2$: loop ends. `arr[2] = 30` → `{10, 20, 30, 40, 50}`

Time Complexity: $O(n)$ worst case (inserting at beginning shifts all elements).
Best case: $O(1)$ (inserting at the end — no shifting needed).

1.3.5 Prefix Sum

The **prefix sum** technique is a powerful optimization. It preprocesses the array once ($O(n)$) so that you can answer any "sum of subarray" query in $O(1)$.

When to Use Prefix Sum

Use prefix sum whenever a problem asks you to find the **sum of elements between two indices** multiple times. Without prefix sum, each query takes $O(n)$. With prefix sum, preprocessing takes $O(n)$ but each query takes $O(1)$.

```c
// Build prefix sum
void buildPrefix(int arr[], int prefix[], int n) {
    prefix[0] = arr[0];
    for (int i = 1; i < n; i++) {
        prefix[i] = prefix[i - 1] + arr[i];
    }
}
// Sum of arr[l..r] = prefix[r] - (l > 0 ? prefix[l-1] : 0)
```

Dry Run with `arr = {1, 2, 3, 4, 5}`:

Index	0	1	2	3	4
arr	1	2	3	4	5
prefix	1	3	6	10	15

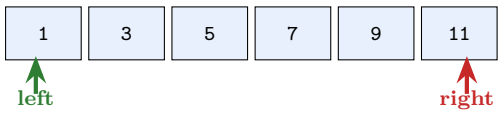
Sum of `arr[1..3]` = `prefix[3] - prefix[0]` $= 10 - 1 = 9$ (i.e., $2 + 3 + 4$).

1.3.6 Two Pointer Technique (Basic Introduction)

The **two pointer** technique uses two pointers (one at the start, one at the end) and moves them inward. This is useful for problems on **sorted arrays** and reduces $O(n^2)$ brute force to $O(n)$.

Real-Life Analogy

Imagine two people searching a sorted bookshelf from **opposite ends**. One starts from the smallest book, the other from the largest. They move toward each other, eliminating half the possibilities with each step. That is the two pointer technique!



Move toward each other

```c
// Check if a sorted array has a pair with given sum
int hasPairSum(int arr[], int n, int target) {
    int left = 0, right = n - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return 1; // Found!
        else if (sum < target) left++; // Need bigger sum
        else right--; // Need smaller sum
    }
    return 0; // No pair found
}
```

Dry Run with `arr = {1, 3, 5, 7, 9}`, target = 8:

Step	left	right	Sum	Action	
1	0 (val=1)	4 (val=9)	10	$10 > 8$, right--	
2	0 (val=1)	3 (val=7)	8	**Found!** Return 1	

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

Interview Tip

Two Pointer is a very common interview pattern. Practice it with: pair sum, removing duplicates, container with most water, and trapping rain water. The key insight: if the array is sorted, two pointers can replace a nested loop.

Practice Problems:

1. Find the maximum and minimum element of an array.
2. Rotate an array by k positions.
3. Find the subarray with maximum sum (Kadane's Algorithm).
4. Move all zeros to the end of an array without changing the order of non-zero elements.

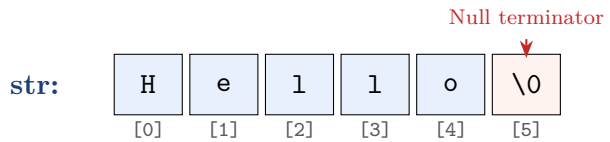
1.4 Strings

1.4.1 What is a String?

In C, a **string** is an array of characters terminated by a special null character `'\0'`. This null character marks the end of the string — without it, C would not know where the string stops.

Real-Life Analogy

Think of a **train of wagons**. Each wagon carries one letter. The last wagon is always a **red caboose** (the null terminator `'\0'`) that signals "the train ends here." If the caboose is missing, the train keeps running off the tracks!




```c
char name[] = "Hello";
// Stored as: 'H','e','l','l','o','\0'
// Array size = 6 (5 characters + 1 null terminator)
```

Common Mistake

When declaring a string as `char str[5] = "Hello"`, you get a **buffer overflow** because `"Hello"` needs 6 bytes (5 chars + null terminator). Always allocate one extra byte for `'\0'`!

1.4.2 String Traversal

To traverse a string, we loop until we hit the null character:

```c
#include <stdio.h>
void printString(char str[]) {
    for (int i = 0; str[i] != '\0'; i++) {
        printf("%c", str[i]);
    }
    printf("\n");
}
```

Why `'\0'` check? Unlike arrays where you pass the size, strings use the null terminator as the built-in "stop sign." This is why we check `str[i] != '\0'` instead of `i < n`.

1.4.3 Palindrome Checking

A **palindrome** reads the same forwards and backwards. Examples: "madam", "racecar", "level".

Approach: Use two pointers — one at the start, one at the end. Compare characters moving inward. If all pairs match, it is a palindrome.

```c
#include <string.h>
int isPalindrome(char str[]) {
    int left = 0, right = strlen(str) - 1;
    while (left < right) {
        if (str[left] != str[right])
            return 0; // Not a palindrome
        left++;
        right--;
    }
    return 1; // Is a palindrome
}
```

Dry Run with `"madam"`:

Step	left	right	Compare	Result	
1	0 (m)	4 (m)	m == m	Match ✓	
2	1 (a)	3 (a)	a == a	Match ✓	
3	2 (d)	2 (d)	Loop ends	Palindrome!	

Time: $O(n/2) = O(n)$ **Space:** $O(1)$

1.4.4 Frequency Counting

Counting how many times each character appears is a fundamental string technique. We use an integer array of size 26 (for lowercase a-z) as a frequency table.

The Frequency Array Trick

The expression `str[i] - 'a'` converts a character to an index: `'a'` → 0, `'b'` → 1, ..., `'z'` → 25. This is because characters are stored as ASCII numbers, and subtracting `'a'` gives the offset from 'a'.

```c
void charFrequency(char str[]) {
    int freq[26] = {0}; // Initialize all counts to 0
    for (int i = 0; str[i] != '\0'; i++) {
        freq[str[i] - 'a']++; // Increment count for this char
    }
    for (int i = 0; i < 26; i++) {
```

```
        if (freq[i] > 0)
            printf("%c: %d\n", i + 'a', freq[i]);
    }
}
```

1.4.5 Basic Pattern Matching

Problem: Given a text and a pattern, find if (and where) the pattern appears in the text.

The **naïve approach** checks every possible starting position in the text:

```
#include <string.h>
int naiveSearch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m) return i; // Pattern found at index i
    }
    return -1; // Not found
}
```

Time Complexity: $O(n \times m)$ worst case (but often faster in practice).

Interview Tip

String problems are very common in interviews. Master these patterns: (1) frequency counting with arrays, (2) two-pointer palindrome check, (3) sliding window for substring problems. These three techniques solve 80% of string interview questions.

Practice Problems:

1. Check if two strings are anagrams (same characters, different order).
2. Count vowels and consonants in a string.
3. Reverse a string without using library functions.
4. Find the first non-repeating character.

1.5 Recursion (Important Foundation)

1.5.1 What is Recursion?

Recursion is when a function calls itself to solve a smaller version of the same problem. Instead of solving a big problem all at once, you break it into a smaller piece and solve that first.

Real-Life Analogy

Imagine you are standing in a queue and want to know your position. You ask the person in front of you: "What is your position?" They ask the person in front of *them*, and so on, until the first person says "I am number 1." Then each person adds 1 and passes the answer back. That chain of "ask, wait, receive" is recursion!

The key idea: A recursive solution has two parts:

1. **Break the problem** into a smaller version of itself.
2. **Combine the result** of the smaller problem to get the answer for the bigger one.

1.5.2 Base Case & Recursive Case

Every recursive function needs:

- **Base Case:** The simplest version of the problem that can be answered directly, without further recursion. This is the "stopping condition."
- **Recursive Case:** The function calls itself with a *smaller* or *simpler* input, making progress toward the base case.

Common Mistake

Forgetting the base case causes **infinite recursion**, which fills up the call stack until your program crashes with a "stack overflow" error. **Always write the base case first!** Before writing any recursive code, ask yourself: "What is the simplest case I can answer directly?"

Three Questions for Every Recursive Problem

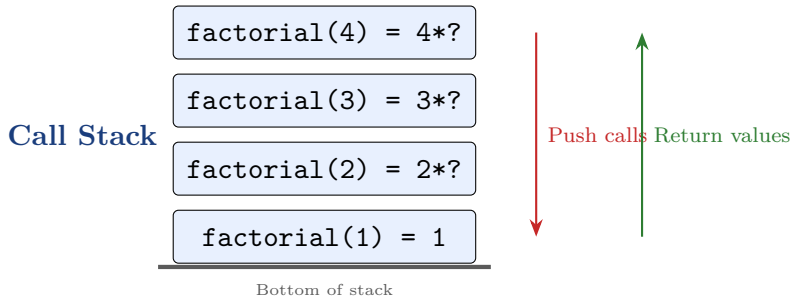
Before writing recursive code, always answer these three questions:

1. **What is the base case?** (When do I stop?)
2. **What is the recursive step?** (How do I make the problem smaller?)
3. **Does each call make progress toward the base case?** (Will I eventually stop?)

If you can answer all three, you can write the recursive solution.

1.5.3 Stack Memory Concept

When a function calls itself, the computer does **not** throw away the current function. Instead, it "pauses" the current call and pushes it onto the **call stack**. When the base case is reached, calls return one by one — this is called **unwinding**.



Each stack frame stores: the function's local variables, the parameter values, and where to return to. This is why deep recursion can cause stack overflow — each call uses memory.

1.5.4 Classic Problem: Factorial

$n! = n \times (n-1) \times \ldots \times 1$, and $0! = 1$.

Recursive thinking: $factorial(n) = n \times factorial(n-1)$, and $factorial(0) = 1$ (base case).

```c
#include <stdio.h>

int factorial(int n) {
    if (n == 0) return 1; // Base case
    return n * factorial(n - 1); // Recursive case
}

int main() {
    printf("5! = %d\n", factorial(5)); // Output: 120
    return 0;
}
```

Dry Run for `factorial(4)`:

Call	Expression	Status	
`factorial(4)`	$4\times$ `factorial(3)`	Waiting...	
`factorial(3)`	$3\times$ `factorial(2)`	Waiting...	
`factorial(2)`	$2\times$ `factorial(1)`	Waiting...	
`factorial(1)`	$1\times$ `factorial(0)`	Waiting...	
`factorial(0)`	returns **1**	Base case!	
Unwinding: $1 \to 1 \to 2 \to 6 \to 24$			

Time Complexity: $O(n)$ — exactly $n + 1$ function calls.
Space Complexity: $O(n)$ — the call stack grows n levels deep.

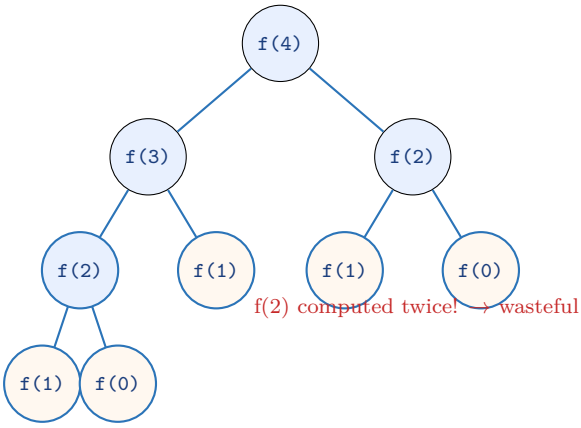
1.5.5 Classic Problem: Fibonacci

The Fibonacci sequence: $0, 1, 1, 2, 3, 5, 8, 13, \ldots$ Each number is the sum of the two before it.

$F(0) = 0,\ F(1) = 1,\ F(n) = F(n-1) + F(n-2)$

```c
int fibonacci(int n) {
    if (n <= 1) return n; // Base case: F(0)=0, F(1)=1
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Time Complexity: $O(2^n)$ — very slow! This is because the same subproblems are solved again and again. We will fix this with **Dynamic Programming** in Chapter 4.



Common Mistake

Naïve recursive Fibonacci is one of the most common examples of **overlapping subproblems**. Notice that `f(2)` is computed twice in the tree above. For `f(40)`, billions of redundant calculations occur. Never use naïve recursion for Fibonacci in production code!

1.5.6 Classic Problem: Sum of Digits

Problem: Find the sum of all digits of a number. E.g., $digitSum(123) = 1 + 2 + 3 = 6$.

Recursive thinking: The last digit is `n % 10`. The remaining number is `n / 10`. Base case: when $n = 0$, the sum is 0.

```c
int sumOfDigits(int n) {
    if (n == 0) return 0; // Base case
```

```
    return (n % 10) + sumOfDigits(n / 10); // Last digit + rest
}
// sumOfDigits(123) = 3 + sumOfDigits(12)
// = 3 + 2 + sumOfDigits(1)
// = 3 + 2 + 1 + sumOfDigits(0) = 6
```

1.5.7 Classic Problem: Reverse an Array

Approach: Swap the first and last elements, then recursively reverse the subarray in between.

```
void reverseArray(int arr[], int left, int right) {
    if (left >= right) return; // Base case: 0 or 1 element
    // Swap the two ends
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    // Recurse on the inner portion
    reverseArray(arr, left + 1, right - 1);
}
```

Dry Run with `{1, 2, 3, 4, 5}`:

1. Swap arr[0] and arr[4]: `{5, 2, 3, 4, 1}`
2. Swap arr[1] and arr[3]: `{5, 4, 3, 2, 1}`
3. $left = 2, right = 2$: base case, stop. Result: `{5, 4, 3, 2, 1}`

Practice Problems:

1. Print numbers from n to 1 using recursion (no loops).
2. Check if a string is a palindrome using recursion.
3. Calculate power x^n using recursion (optimize to $O(\log n)$).
4. Print all subsequences of a string.

Interview Tip

Interviewers love recursion questions. The key insight they look for: **Can you identify the subproblem?** If you can express the solution for n in terms of the solution for $n - 1$ (or some smaller input), you have the recursive formula. Practice "thinking recursively" — it is a skill that improves with repetition.

Outcome

Student understands how to analyze problems using Big-O notation, evaluate algorithmic complexity, and build foundational logic using arrays, strings, and recursion. The student can write basic C programs, trace through recursive calls on paper, and identify when to use techniques like two-pointer and prefix sum.

Chapter 2

Linear Data Structures

2.1 Linked List

2.1.1 What is a Linked List?

A **linked list** is a linear data structure where elements (called **nodes**) are connected using pointers. Unlike arrays, the elements are *not* stored in contiguous memory — they can be scattered anywhere in memory, with each node "pointing" to the next one.

Real-Life Analogy

Think of a **treasure hunt**. Each clue card tells you where to find the *next* clue. You must follow the chain from the first clue to find the treasure. You cannot jump directly to clue #5 — you must go through clues 1, 2, 3, 4 first. A linked list works the same way.

Why not just use arrays? Arrays have two big limitations:

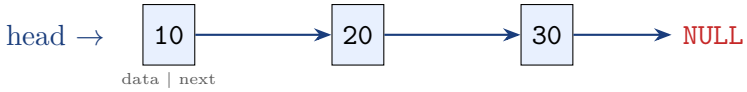
- **Fixed size:** Once created, you cannot easily grow an array.
- **Expensive insertion/deletion:** Inserting at the middle requires shifting all subsequent elements — $O(n)$.

Linked lists solve both problems: they grow dynamically (one node at a time) and insertion/deletion at a known position takes $O(1)$.

Array vs. Linked List — When to Use Which

Operation	Array	Linked List
Access by index	$O(1)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Insert at end	$O(1)$*	$O(n)$**
Search	$O(n)$	$O(n)$
Memory	Contiguous	Scattered

*Amortized for dynamic arrays **$O(1)$ if you maintain a tail pointer



Each node contains two parts: (1) the **data** it stores, and (2) a **pointer** to the next node. The last node's pointer is `NULL`, meaning "no more nodes."

2.1.2 Node Structure in C

```c
struct Node {
    int data; // The value stored in this node
    struct Node *next; // Pointer to the next node
};
```

Why `struct Node *next`? This is a *self-referential structure* — the structure contains a pointer to another instance of itself. This is how nodes link together.

2.1.3 Singly Linked List — Insert at Beginning

Step-by-step logic:

1. Create a new node using `malloc`.
2. Set the new node's data to the desired value.
3. Point the new node's `next` to the current head.
4. Update head to point to the new node.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

// Insert a new node at the beginning
struct Node* insertAtHead(struct Node *head, int val) {
    struct Node *newNode = (struct Node *)
        malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = head; // Point to old head
    return newNode; // New node becomes the head
}

// Print the linked list
```

```c
void printList(struct Node *head) {
    struct Node *temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head = NULL;
    head = insertAtHead(head, 30);
    head = insertAtHead(head, 20);
    head = insertAtHead(head, 10);
    printList(head); // Output: 10 -> 20 -> 30 -> NULL
    return 0;
}
```

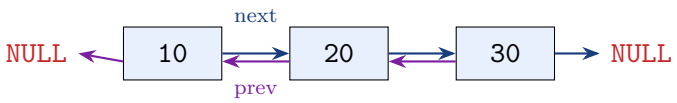
Time Complexity: $O(1)$ for insertion at head — no traversal needed!

Common Mistake

Always check that `malloc` returns a non-NULL pointer. In production code, `malloc` can fail if memory is exhausted. Also, always `free` nodes when you delete them to avoid **memory leaks**.

2.1.4 Doubly Linked List

Each node has pointers to **both** the next and the previous node. This allows traversal in both directions.




```c
struct DNode {
    int data;
    struct DNode *prev; // Pointer to previous node
    struct DNode *next; // Pointer to next node
};
```

Advantage: You can traverse in both directions — useful for operations like "delete a node when you have a pointer to it" ($O(1)$ with doubly linked, $O(n)$ with singly linked).
Disadvantage: Uses extra memory for the `prev` pointer and insertion/deletion code is more complex.

2.1.5 Circular Linked List

The last node points back to the first node instead of `NULL`. This creates a loop — useful for problems like the Josephus problem and round-robin scheduling.

2.1.6 Reverse a Linked List

This is one of the most popular interview questions. The idea is to reverse all the arrows.

Step-by-step logic:

1. Maintain three pointers: `prev` (starts at NULL), `curr` (starts at head), `next` (temporary).
2. For each node: save `next`, reverse the link (`curr->next = prev`), move `prev` and `curr` forward.
3. When done, `prev` is the new head.

```c
struct Node* reverseList(struct Node *head) {
    struct Node *prev = NULL;
    struct Node *curr = head;
    struct Node *next = NULL;
    while (curr != NULL) {
        next = curr->next; // Save next
        curr->next = prev; // Reverse the link
        prev = curr; // Move prev forward
        curr = next; // Move curr forward
    }
    return prev; // prev is the new head
}
```

Dry Run with list $1 \to 2 \to 3$:

Step	prev	curr	next	
0	NULL	1	—	
1	1	2	3	
2	2	3	NULL	
3	3	NULL	—	

Result: $3 \to 2 \to 1 \to$ NULL. New head = 3.

2.1.7 Detect Cycle — Floyd's Algorithm

Use two pointers: **slow** (moves 1 step) and **fast** (moves 2 steps). If they meet, there is a cycle.

```c
int hasCycle(struct Node *head) {
    struct Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1; // Cycle detected
    }
    return 0; // No cycle
}
```

Time: $O(n)$ **Space:** $O(1)$

Interview Tip

Floyd's Cycle Detection is asked in almost every interview. Follow-up questions include: "Find the start of the cycle" and "Find the length of the cycle."

Practice Problems:

1. Find the middle element of a linked list in one pass.
2. Delete the n-th node from the end.
3. Merge two sorted linked lists.

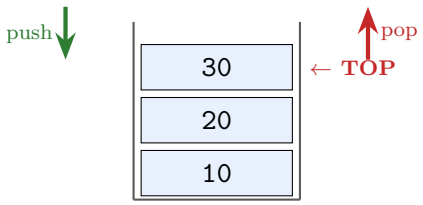
2.2 Stack

2.2.1 What is a Stack?

A **stack** is a linear data structure that follows **LIFO** — Last In, First Out. The element added *most recently* is the first one to be removed.

Real-Life Analogy

Think of a **stack of plates** in a cafeteria. You always place a new plate on top and remove the top plate first. You cannot remove a plate from the middle without removing all plates above it. The bottom plate was added first but will be removed last — that is LIFO!



Core Operations:

- **push(x)** — add element x to the top. Time: $O(1)$.
- **pop()** — remove and return the top element. Time: $O(1)$.
- **peek()** — look at the top element without removing it. Time: $O(1)$.
- **isEmpty()** — check if the stack has no elements.

When to Use a Stack

Use a stack whenever you need to **reverse order**, **undo operations**, or process things in a **last-in-first-out** manner. Common uses: function call stack, undo/redo in editors, parenthesis matching, expression evaluation, DFS traversal.

2.2.2 Stack Using Array

```c
#include <stdio.h>
#define MAX 100

struct Stack {
    int arr[MAX];
    int top;
};

void init(struct Stack *s) { s->top = -1; }

int isEmpty(struct Stack *s) { return s->top == -1; }

int isFull(struct Stack *s) { return s->top == MAX - 1; }

void push(struct Stack *s, int val) {
    if (isFull(s)) { printf("Stack Overflow!\n"); return; }
    s->arr[++(s->top)] = val;
}

int pop(struct Stack *s) {
    if (isEmpty(s)) { printf("Stack Underflow!\n"); return -1; }
    return s->arr[(s->top)--];
}

int peek(struct Stack *s) {
    if (isEmpty(s)) return -1;
    return s->arr[s->top];
}
```

2.2.3 Stack Using Linked List

```c
struct StackNode {
    int data;
    struct StackNode *next;
};

void push(struct StackNode **top, int val) {
    struct StackNode *node = (struct StackNode *)
        malloc(sizeof(struct StackNode));
    node->data = val;
    node->next = *top;
    *top = node;
}

int pop(struct StackNode **top) {
    if (*top == NULL) { printf("Underflow!\n"); return -1; }
    struct StackNode *temp = *top;
    int val = temp->data;
    *top = temp->next;
    free(temp);
    return val;
}
```

2.2.4 Application: Parenthesis Checking

Check if brackets are balanced: `{[()]}` is valid, `{[(]}` is not.

```c
int isBalanced(char expr[]) {
    struct Stack s;
    init(&s);
    for (int i = 0; expr[i] != '\0'; i++) {
        char ch = expr[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push(&s, ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty(&s)) return 0;
            char top = pop(&s);
            if ((ch == ')' && top != '(') ||
                (ch == '}' && top != '{') ||
                (ch == ']' && top != '['))
                return 0;
        }
```

```
    }
    return isEmpty(&s); // Valid if stack is empty
}
```

2.2.5 Application: Infix to Postfix

Infix: `A + B * C` → Postfix: `A B C * +`

The algorithm uses a stack to hold operators and respects precedence.

2.2.6 Application: Next Greater Element

For each element, find the next element that is greater. Uses a **monotonic stack**.

```
void nextGreater(int arr[], int n, int result[]) {
    struct Stack s;
    init(&s);
    for (int i = n - 1; i >= 0; i--) {
        while (!isEmpty(&s) && peek(&s) <= arr[i])
            pop(&s);
        result[i] = isEmpty(&s) ? -1 : peek(&s);
        push(&s, arr[i]);
    }
}
// Input: [4, 5, 2, 25]
// Output: [5, 25, 25, -1]
```

Interview Tip

Next Greater Element and its variants (next smaller, next greater circular) appear frequently in interviews at Amazon, Google, and Microsoft.

Practice Problems:

1. Evaluate a postfix expression.
2. Sort a stack using another stack.
3. Implement a min-stack (get minimum in $O(1)$).

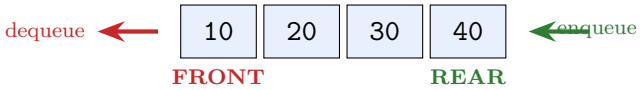
2.3 Queue

2.3.1 What is a Queue?

A **queue** follows **FIFO** — First In, First Out. The element added *first* is the first one to be removed. This is the opposite of a stack.

Real-Life Analogy

Think of a **line at a ticket counter**. The first person who arrives gets served first. New arrivals join at the back (rear), and people leave from the front. Cutting the line is not allowed — that would violate FIFO!



Core Operations:

- **enqueue(x)** — add element x at the rear. Time: $O(1)$.
- **dequeue()** — remove and return the front element. Time: $O(1)$.
- **front()** — look at the front element without removing it.

Stack vs Queue — Quick Comparison

Stack = LIFO (plates). Last added, first removed.
Queue = FIFO (ticket line). First added, first removed.
Stack is used for DFS and undo operations. Queue is used for BFS and scheduling.

2.3.2 Simple Queue Using Array

```c
#define MAX 100
struct Queue {
    int arr[MAX];
    int front, rear;
};

void init(struct Queue *q) { q->front = 0; q->rear = -1; }

void enqueue(struct Queue *q, int val) {
    if (q->rear == MAX - 1) { printf("Queue Full!\n"); return; }
    q->arr[++(q->rear)] = val;
}

int dequeue(struct Queue *q) {
    if (q->front > q->rear) { printf("Queue Empty!\n"); return -1; }
    return q->arr[(q->front)++];
```

```
}
```

2.3.3 Circular Queue

In a circular queue, when `rear` reaches the end, it wraps around to the beginning. This avoids wasting space.

```c
void enqueueCircular(struct Queue *q, int val) {
    int nextRear = (q->rear + 1) % MAX;
    if (nextRear == q->front) { printf("Full!\n"); return; }
    q->rear = nextRear;
    q->arr[q->rear] = val;
}
```

2.3.4 Deque (Double-Ended Queue)

Allows insertion and deletion from **both ends**. Useful for problems like Sliding Window Maximum.

2.3.5 Priority Queue (Introduction)

Elements are dequeued based on **priority**, not arrival order. We will implement this using a **Heap** in Module 3.

2.3.6 Application: Sliding Window Maximum

Find the maximum in every window of size k in an array. Uses a deque for $O(n)$ solution.

Interview Tip

Queue is fundamental to **BFS** (Breadth-First Search) in graphs and trees. Understanding queues is a prerequisite for graph algorithms.

Practice Problems:

1. Implement a queue using two stacks.
2. Generate binary numbers from 1 to n using a queue.
3. Implement a circular queue with all operations.

2.4 Hashing

2.4.1 What is Hashing?

Hashing is a technique to map data to a fixed-size table for **near-instant lookup**. A **hash function** converts a key into an array index. The result is called a **hash table** — a data structure that achieves $O(1)$ average-case search, insert, and delete.

Real-Life Analogy

Think of a **library catalog**. Instead of searching every shelf for a book, you look up the book's catalog number and go directly to the right shelf. The catalog number is computed from the book's title — that is the hash function. Sometimes two books map to the same shelf (collision), so you store them in a list on that shelf.

How it works, step by step:

1. You have an array of size T (the hash table).
2. To store key k: compute $index = hash(k)\%T$.
3. Store k at `table[index]`.
4. To find k: compute the same index and look there. $O(1)$!

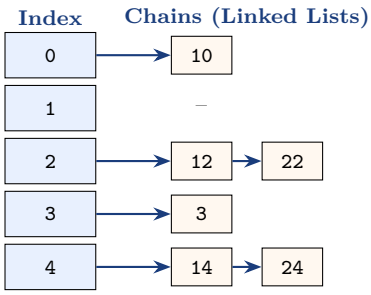
2.4.2 Hash Table Basics

```c
#define TABLE_SIZE 10

int hashFunction(int key) {
    return key % TABLE_SIZE; // Simple modulo hash
}
```

2.4.3 Collision Handling

When two keys map to the same index, we have a **collision**. Example: with table size 10, both key 15 and key 25 hash to index 5. Two main solutions:



12 and 22 both hash to index 2 → collision resolved by chaining

1. **Chaining:** Each index stores a linked list of all elements that hash to that index.

Simple and handles unlimited collisions.

2. Open Addressing: If a slot is taken, probe the next slot (linear probing). No extra memory for linked lists, but can cause clustering.

```c
// Simple hash table with linear probing
int table[TABLE_SIZE];
int occupied[TABLE_SIZE]; // 0 = empty, 1 = occupied

void initTable() {
    for (int i = 0; i < TABLE_SIZE; i++) occupied[i] = 0;
}

void insert(int key) {
    int idx = hashFunction(key);
    while (occupied[idx]) {
        idx = (idx + 1) % TABLE_SIZE; // Linear probe
    }
    table[idx] = key;
    occupied[idx] = 1;
}

int search(int key) {
    int idx = hashFunction(key);
    int start = idx;
    while (occupied[idx]) {
        if (table[idx] == key) return idx; // Found
        idx = (idx + 1) % TABLE_SIZE;
        if (idx == start) break; // Full circle
    }
    return -1; // Not found
}
```

2.4.4 Frequency-Based Problems

```c
// Count frequency of each element in an array
void countFrequency(int arr[], int n) {
    int hash[1000] = {0}; // Assuming values 0-999
    for (int i = 0; i < n; i++) {
        hash[arr[i]]++;
    }
    for (int i = 0; i < 1000; i++) {
        if (hash[i] > 0)
            printf("%d appears %d times\n", i, hash[i]);
```

```
    }
}
```

Time Complexity: $O(1)$ average for insert/search. $O(n)$ worst case with many collisions.

Common Mistake

A bad hash function causes many collisions, degrading performance to $O(n)$. Always choose a hash function that distributes keys uniformly.

Practice Problems:

1. Find the first non-repeating character in a string.
2. Check if two arrays are equal (contain same elements).
3. Find the number of pairs with a given sum using hashing.

Outcome

Student understands pointer-based data structures, memory management with `malloc`/`free`, and can implement linked lists, stacks, queues, and hash tables in C. The student can apply these structures to solve real-world problems.

Chapter 3

Non-Linear Data Structures

3.1 Trees

3.1.1 What is a Tree?

A **tree** is a non-linear data structure made of **nodes** connected by **edges**. Unlike arrays, linked lists, stacks, and queues (which are all linear), trees organize data *hierarchically* — perfect for modeling relationships like folders on a computer, organizational charts, or HTML DOM.

Every tree has one special node called the **root** (the topmost node). Every other node has exactly one **parent** and zero or more **children**.

Real-Life Analogy

Think of a **family tree**. There is one ancestor at the top (root). Each person can have children, and each child has exactly one parent. The people with no children are called "leaves." You can trace a path from the root to any family member.

Key Terminology:

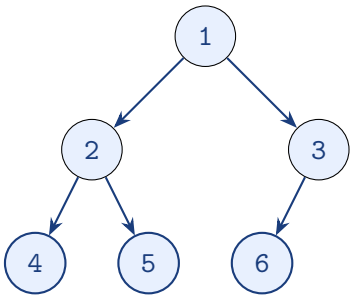
- **Root:** The topmost node (has no parent).
- **Leaf:** A node with no children.
- **Edge:** Connection between a parent and a child.
- **Height:** Longest path from root to any leaf (counted in edges).
- **Depth of a node:** Number of edges from the root to that node.
- **Subtree:** Any node and all its descendants form a subtree.

Types of Binary Trees

- **Full Binary Tree:** Every node has 0 or 2 children (no node has just 1 child).
- **Complete Binary Tree:** All levels are fully filled except possibly the last, which is filled from left to right. *Heaps use this structure.*
- **Perfect Binary Tree:** All internal nodes have 2 children and all leaves are at the same level.
- **Balanced Binary Tree:** Height difference between left and right subtrees of any node is at most 1. Guarantees $O(\log n)$ operations.

3.1.2 Binary Tree

A **binary tree** is a tree where each node has **at most two children**: left and right.




```c
struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
};

struct TreeNode* createNode(int val) {
    struct TreeNode *node = (struct TreeNode *)
        malloc(sizeof(struct TreeNode));
    node->data = val;
    node->left = NULL;
    node->right = NULL;
    return node;
}
```

3.1.3 Tree Traversals

Traversal means "visiting every node exactly once." The order in which we visit nodes defines the traversal type. Understanding traversals is essential — they are the foundation for almost every tree algorithm.

Mnemonic: Where Does the Root Go?

The name tells you when to visit the **root**:

- **In**order — root in the *middle*: Left → **Root** → Right
- **Pre**order — root *first* (pre = before): **Root** → Left → Right
- **Post**order — root *last* (post = after): Left → Right → **Root**

This mnemonic works for every tree — just remember where the Root goes!

There are four standard ways to visit every node:

1. Inorder (Left → Root → Right):

```c
void inorder(struct TreeNode *root) {
    if (root == NULL) return;
    inorder(root->left); // Visit left subtree
    printf("%d ", root->data); // Visit root
    inorder(root->right); // Visit right subtree
}
// For the tree above: 4 2 5 1 6 3
```

2. Preorder (Root → Left → Right):

```c
void preorder(struct TreeNode *root) {
    if (root == NULL) return;
    printf("%d ", root->data); // Visit root first
    preorder(root->left);
    preorder(root->right);
}
// Output: 1 2 4 5 3 6
```

3. Postorder (Left → Right → Root):

```c
void postorder(struct TreeNode *root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data); // Visit root last
}
// Output: 4 5 2 6 3 1
```

4. Level Order (Breadth-First): Visit level by level using a queue.

```c
// Level order requires a queue €"â conceptual approach:
// 1. Enqueue root
// 2. While queue is not empty:
// a. Dequeue node, print it
```

```
// b. Enqueue its left and right children
```

3.1.4 Height, Depth, and Leaf Nodes

```c
int height(struct TreeNode *root) {
    if (root == NULL) return -1; // or 0, depending on convention
    int leftH = height(root->left);
    int rightH = height(root->right);
    return 1 + (leftH > rightH ? leftH : rightH);
}

int countLeaves(struct TreeNode *root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
```

3.1.5 Diameter of Tree

The **diameter** is the longest path between any two nodes (in terms of edges).

```c
int diameter(struct TreeNode *root, int *maxDia) {
    if (root == NULL) return 0;
    int leftH = diameter(root->left, maxDia);
    int rightH = diameter(root->right, maxDia);
    int currentDia = leftH + rightH;
    if (currentDia > *maxDia) *maxDia = currentDia;
    return 1 + (leftH > rightH ? leftH : rightH);
}
```

Interview Tip

Tree traversals are fundamental interview questions. Be ready to write all four from memory. Follow-up: "Do it iteratively using a stack."

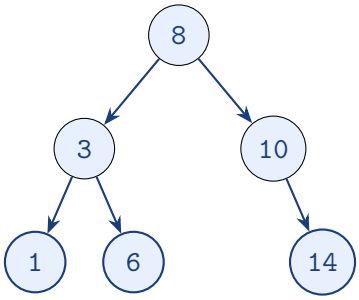
Practice Problems:

1. Check if two trees are identical.
2. Find the mirror/invert of a binary tree.
3. Check if a binary tree is balanced.

3.2 Binary Search Tree (BST)

3.2.1 What is a BST?

A **BST** is a binary tree where for every node: all values in the **left** subtree are **smaller**, and all values in the **right** subtree are **larger**.



3.2.2 Search in BST

Searching in a BST is efficient because at each node, we eliminate half the tree. If the key is smaller than the current node, go left. If larger, go right. If equal, found!

```c
struct TreeNode* search(struct TreeNode *root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data)
        return search(root->left, key); // Go left
    else
        return search(root->right, key); // Go right
}
// Time: O(h) where h = height. O(log n) average, O(n) worst.
```

Dry Run: Search for key = 6 in the BST $[8, 3, 10, 1, 6, 14]$:

Step	Current Node	Decision	
1	8	$6 < 8$, go left	
2	3	$6 > 3$, go right	
3	6	$6 == 6$, found!	

Only 3 comparisons instead of checking all 6 nodes — that is the power of BST!

3.2.3 Insert in BST

```c
struct TreeNode* insert(struct TreeNode *root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
```

```
        root->right = insert(root->right, val);
    return root;
}
```

3.2.4 Valid BST Check

```
#include <limits.h>
int isValidBST(struct TreeNode *root, long min, long max) {
    if (root == NULL) return 1;
    if (root->data <= min || root->data >= max) return 0;
    return isValidBST(root->left, min, root->data) &&
           isValidBST(root->right, root->data, max);
}
// Call: isValidBST(root, LONG_MIN, LONG_MAX)
```

3.2.5 Lowest Common Ancestor (LCA)

```
struct TreeNode* lcaBST(struct TreeNode *root, int p, int q) {
    if (root == NULL) return NULL;
    if (p < root->data && q < root->data)
        return lcaBST(root->left, p, q); // Both left
    if (p > root->data && q > root->data)
        return lcaBST(root->right, p, q); // Both right
    return root; // Split point = LCA
}
```

Common Mistake

Do not confuse LCA in a *BST* (use BST property, $O(\log n)$) with LCA in a general *binary tree* (different algorithm, $O(n)$). Interviewers will notice.

Practice Problems:

1. Find the kth smallest element in a BST.
2. Delete a node from a BST.
3. Convert a sorted array to a balanced BST.

3.3 Heap

3.3.1 What is a Heap?

A **heap** is a *complete binary tree* that satisfies the **heap property**. The brilliant insight is that because it is complete, we can store it in a simple **array** — no pointers needed!

- **Min Heap:** Parent $\leq$ Children (root is the smallest).
- **Max Heap:** Parent $\geq$ Children (root is the largest).

Real-Life Analogy

Think of a **company hierarchy**. In a Max Heap, the CEO (root) always has a higher salary than any manager below. Each manager earns more than their direct reports. The CEO is always at the top — you can find the highest-paid person in $O(1)$!

Heap Array Mapping — No Pointers Needed!

For a node at index i (0-indexed):

- **Parent** $= \lfloor (i-1)/2 \rfloor$
- **Left Child** $= 2i + 1$
- **Right Child** $= 2i + 2$

Example array: `[10, 20, 30, 40, 50]`
Node 10 (index 0) has children 20 (index 1) and 30 (index 2).
Node 20 (index 1) has children 40 (index 3) and 50 (index 4).
This mapping lets us navigate the tree using simple arithmetic — no `left`/`right` pointers!

3.3.2 Heap Operations

Insert (Heapify Up):

```c
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

void insertMinHeap(int heap[], int *size, int val) {
    heap[*size] = val;
    int i = *size;
    (*size)++;
    // Heapify up: compare with parent
    while (i > 0 && heap[i] < heap[(i - 1) / 2]) {
        swap(&heap[i], &heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}
```

Extract Min (Heapify Down):

```c
int extractMin(int heap[], int *size) {
    if (*size == 0) return -1;
    int min = heap[0];
    heap[0] = heap[--(*size)]; // Move last to root
    // Heapify down
    int i = 0;
    while (1) {
        int smallest = i;
        int left = 2 * i + 1, right = 2 * i + 2;
        if (left < *size && heap[left] < heap[smallest])
            smallest = left;
        if (right < *size && heap[right] < heap[smallest])
            smallest = right;
        if (smallest == i) break;
        swap(&heap[i], &heap[smallest]);
        i = smallest;
    }
    return min;
}
```

Time: Insert $O(\log n)$, Extract $O(\log n)$, Build heap $O(n)$.

3.3.3 Heap Sort

Build a max heap from the array, then repeatedly extract the maximum:

```c
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // Extract elements one by one
```

```
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]); // Move max to end
        heapify(arr, i, 0); // Heapify reduced heap
    }
}
```

Time: $O(n \log n)$ **Space:** $O(1)$ (in-place)

3.3.4 Kth Largest Element

Use a min heap of size k. The root gives the k-th largest.

```
// Approach: maintain a min heap of size k
// Insert all elements; if heap size > k, remove min
// After processing all elements, root = kth largest
```

Interview Tip

"Find the Kth largest/smallest element" is a top-10 interview question. Know both the heap approach ($O(n \log k)$) and the quickselect approach ($O(n)$ average).

Practice Problems:

1. Sort a nearly sorted (k-sorted) array using a heap.
2. Merge k sorted arrays using a min heap.
3. Find the median of a data stream (use two heaps).

3.4 Graphs

3.4.1 What is a Graph?

A **graph** $G = (V, E)$ is a collection of **vertices** (nodes) V connected by **edges** E. While trees have a strict parent-child hierarchy, graphs have *no such restriction* — any node can connect to any other node. This makes graphs the most general data structure for modeling relationships.

Real-Life Analogy

Think of a **city road map**. Cities are vertices. Roads between cities are edges. Some roads are one-way (directed), some are two-way (undirected). Some roads have toll costs (weighted edges). Finding the shortest route from City A to City B is a graph problem!

Types of Graphs:

- **Undirected:** Edges have no direction (friendship is mutual).
- **Directed:** Edges have direction (following someone on Twitter is one-way).
- **Weighted:** Edges have a cost/distance/weight associated with them.
- **Unweighted:** All edges are treated equally.
- **Cyclic:** Contains at least one cycle (a path that starts and ends at the same node).
- **Acyclic:** No cycles. A *Directed Acyclic Graph* (DAG) is common in scheduling problems.

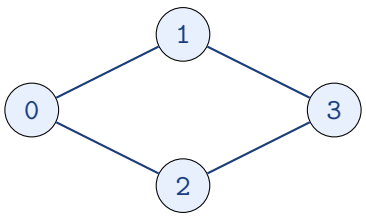
Adjacency Matrix vs. Adjacency List

Property	Matrix	List
Space	$O(V^2)$	$O(V+E)$
Check if edge exists	$O(1)$	$O(\text{degree})$
Find all neighbors	$O(V)$	$O(\text{degree})$
Best for	Dense graphs	Sparse graphs

Most real-world graphs are sparse, so adjacency lists are preferred in practice.

3.4.2 Graph Representation

1. Adjacency Matrix:




```c
#define V 4
int adjMatrix[V][V] = {
    {0, 1, 1, 0}, // 0 connects to 1, 2
    {1, 0, 0, 1}, // 1 connects to 0, 3
    {1, 0, 0, 1}, // 2 connects to 0, 3
    {0, 1, 1, 0} // 3 connects to 1, 2
};
// Space: O(V^2) €"â good for dense graphs
```

2. Adjacency List:

```c
struct AdjNode {
    int vertex;
    struct AdjNode *next;
};
```

```c

struct Graph {
    int numVertices;
    struct AdjNode **adjLists; // Array of linked lists
};

void addEdge(struct Graph *g, int src, int dest) {
    // Add dest to src's list
    struct AdjNode *node = (struct AdjNode *)
        malloc(sizeof(struct AdjNode));
    node->vertex = dest;
    node->next = g->adjLists[src];
    g->adjLists[src] = node;
    // For undirected, also add src to dest's list
    node = (struct AdjNode *)malloc(sizeof(struct AdjNode));
    node->vertex = src;
    node->next = g->adjLists[dest];
    g->adjLists[dest] = node;
}
// Space: O(V + E) €"â good for sparse graphs
```

3.4.3 BFS (Breadth-First Search)

Visit all neighbors at the current depth before moving to the next level. Uses a **queue**.

```c
void bfs(int adjMatrix[V][V], int start) {
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;
    visited[start] = 1;
    queue[rear++] = start;
    while (front < rear) {
        int curr = queue[front++];
        printf("%d ", curr);
        for (int i = 0; i < V; i++) {
            if (adjMatrix[curr][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
}
// Time: O(V + E) with adjacency list
```

3.4.4 DFS (Depth-First Search)

Go as deep as possible before backtracking. Uses **recursion** (or a stack).

```c
void dfs(int adjMatrix[V][V], int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);
    for (int i = 0; i < V; i++) {
        if (adjMatrix[vertex][i] == 1 && !visited[i]) {
            dfs(adjMatrix, i, visited);
        }
    }
}
// Time: O(V + E)
```

3.4.5 Cycle Detection (Basic)

Undirected graph: During DFS, if we visit a node that is already visited and is not the parent, there is a cycle.

Directed graph: Use DFS with three states: unvisited, in-progress, completed.

3.4.6 Connected Components

Run DFS/BFS from each unvisited vertex. Each run discovers one connected component.

```c
int countComponents(int adjMatrix[V][V]) {
    int visited[V] = {0};
    int count = 0;
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(adjMatrix, i, visited);
            count++;
        }
    }
    return count;
}
```

Interview Tip

BFS and DFS are the foundation of almost all graph problems. Master these first. Common follow-ups: shortest path (BFS), topological sort (DFS), and detecting bipartite graphs (BFS with coloring).

Practice Problems:

1. Find whether a path exists between two nodes.
2. Detect a cycle in an undirected graph.
3. Count the number of islands (2D grid BFS/DFS).

Outcome

Student can model, traverse, and solve problems involving hierarchical and network-based data structures, including trees, heaps, and graphs. The student understands BFS, DFS, and their applications.

Chapter 4

Algorithms & Problem-Solving Patterns

4.1 Searching Algorithms

4.1.1 Linear Search

Check every element one by one until you find the target.

```
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i; // Found at index i
    }
    return -1; // Not found
}
```

Time: $O(n)$ **When to use:** Unsorted data, small arrays.

4.1.2 Binary Search

Works on **sorted arrays**. Repeatedly divide the search space in half.

Real-Life Analogy

Think of guessing a number between 1 and 100. Instead of trying 1, 2, 3, ...you guess 50 first. If the answer is "higher," you try 75. Each guess eliminates half the possibilities. That is binary search!

```
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Avoid overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
```

```
}
```

Dry Run with `arr = [2, 5, 8, 12, 16]`, target = 12:

Step	low	high	mid	Result	
1	0	4	2	$8 < 12$, go right	
2	3	4	3	$12 == 12$, found!	

Time: $O(\log n)$ **Space:** $O(1)$

Common Mistake

Use `mid = low + (high - low) / 2` instead of `(low + high) / 2` to avoid integer overflow when `low + high` exceeds `INT_MAX`.

4.1.3 Binary Search on Answer

Apply binary search when the answer lies in a **monotonic search space**. Example: "Find the minimum capacity to ship packages within D days."

```c
// Template: Binary search on answer
int binarySearchOnAnswer(int low, int high) {
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (isFeasible(mid)) {
            ans = mid; // mid works, try smaller
            high = mid - 1;
        } else {
            low = mid + 1; // mid too small
        }
    }
    return ans;
}
```

Interview Tip

Binary Search on Answer is a pattern seen in many hard interview problems. The key insight: if a predicate is monotonic (all TRUEs come before all FALSEs, or vice versa), binary search applies.

Practice Problems:

1. Find the first and last occurrence of an element in a sorted array.
2. Find the square root of a number using binary search.

3. Search in a rotated sorted array.

4.2 Sorting Algorithms

4.2.1 Bubble Sort

Repeatedly compare adjacent elements and swap if they are in the wrong order.

```c
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break; // Already sorted
    }
}
// Time: O(n^2) worst/average, O(n) best (already sorted)
```

4.2.2 Selection Sort

Find the minimum element and place it at the beginning. Repeat for the unsorted portion.

```c
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}
// Time: O(n^2) always. Not stable.
```

4.2.3 Insertion Sort

Build the sorted array one element at a time by inserting each element into its correct position.

```c
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // Shift right
            j--;
        }
        arr[j + 1] = key;
    }
}
// Time: O(n^2) worst, O(n) best (nearly sorted). Stable.
```

4.2.4 Merge Sort

Divide the array into halves, sort each half recursively, then merge them.

```c
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int left[n1], right[n2];
    for (int i = 0; i < n1; i++) left[i] = arr[l + i];
    for (int j = 0; j < n2; j++) right[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else arr[k++] = right[j++];
    }
    while (i < n1) arr[k++] = left[i++];
    while (j < n2) arr[k++] = right[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}
// Time: O(n log n) always. Space: O(n). Stable.
```

4.2.5 Quick Sort

Choose a **pivot**, partition the array so elements smaller than the pivot go left and larger go right, then recurse.

```c
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choose last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
// Time: O(n log n) average, O(n^2) worst. Space: O(log n). Not stable.
```

4.2.6 Comparison of Sorting Techniques

Algorithm	Best	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

Interview Tip

Interviewers expect you to know when to use which sort. **Merge Sort** for guaranteed $O(n \log n)$ and stability. **Quick Sort** for fastest average-case in-place sorting. **Insertion Sort** for small or nearly sorted arrays.

Practice Problems:

1. Sort an array of 0s, 1s, and 2s (Dutch National Flag).
2. Count inversions in an array using Merge Sort.
3. Sort characters of a string.

4.3 Greedy Algorithms

4.3.1 What is a Greedy Algorithm?

A **greedy algorithm** makes the **locally optimal choice** at each step, hoping to reach a globally optimal solution.

Real-Life Analogy

Imagine you want to travel across the country with the least fuel. A greedy approach: at every junction, take the road that covers the most distance per liter. It does not always give the best route overall, but for certain problems it does.

4.3.2 Activity Selection Problem

Given activities with start and end times, select the maximum number of non-overlapping activities.

```c
// Activities sorted by end time
void activitySelection(int start[], int end[], int n) {
    printf("Activity 0: [%d, %d]\n", start[0], end[0]);
    int lastEnd = end[0];
    for (int i = 1; i < n; i++) {
        if (start[i] >= lastEnd) {
            printf("Activity %d: [%d, %d]\n", i, start[i], end[i]);
            lastEnd = end[i];
        }
    }
}
// Greedy choice: always pick the activity that ends earliest.
// Time: O(n log n) for sorting + O(n) for selection.
```

4.3.3 Coin Change (Greedy Case)

Given denominations, find the minimum number of coins to make a given amount. Greedy works for standard denominations (1, 5, 10, 25).

```c
void coinChange(int coins[], int n, int amount) {
```

```
    // coins sorted in descending order
    for (int i = 0; i < n; i++) {
        while (amount >= coins[i]) {
            printf("%d ", coins[i]);
            amount -= coins[i];
        }
    }
}
```

Common Mistake

Greedy does **not** always work for coin change. For denominations like {1, 3, 4} and amount 6, greedy gives 4+1+1=3 coins, but optimal is 3+3=2 coins. Use DP for arbitrary denominations.

4.3.4 Fractional Knapsack

You can take *fractions* of items. Sort by value-to-weight ratio and take greedily.

```
// Assuming items sorted by value/weight ratio (descending)
double fractionalKnapsack(int weights[], int values[],
                          int n, int capacity) {
    double totalValue = 0.0;
    for (int i = 0; i < n; i++) {
        if (weights[i] <= capacity) {
            totalValue += values[i];
            capacity -= weights[i];
        } else {
            totalValue += values[i] * ((double)capacity / weights[i]);
            break;
        }
    }
    return totalValue;
}
```

Practice Problems:

1. Job sequencing with deadlines.
2. Minimum number of platforms required at a railway station.
3. Assign cookies to children (LeetCode 455).

4.4 Recursion to Backtracking

4.4.1 What is Backtracking?

Backtracking is recursion with the ability to **undo choices** when they lead to a dead end. It explores all possibilities but prunes invalid paths early.

Real-Life Analogy

Think of solving a **maze**. You walk forward until you hit a wall (dead end). Then you *backtrack* to the last junction and try a different path. You keep doing this until you find the exit.

4.4.2 Generate All Subsets

```c
void subsets(int arr[], int n, int idx, int current[], int size) {
    // Print current subset
    printf("{ ");
    for (int i = 0; i < size; i++) printf("%d ", current[i]);
    printf("}\n");

    for (int i = idx; i < n; i++) {
        current[size] = arr[i]; // Include arr[i]
        subsets(arr, n, i + 1, current, size + 1); // Recurse
        // Backtrack: arr[i] is automatically excluded
    }
}
// Time: O(2^n) €"â there are 2^n subsets
```

4.4.3 Generate All Permutations

```c
void permute(int arr[], int l, int r) {
    if (l == r) {
        for (int i = 0; i <= r; i++) printf("%d ", arr[i]);
        printf("\n");
        return;
    }
    for (int i = l; i <= r; i++) {
        // Swap arr[l] and arr[i]
        int temp = arr[l]; arr[l] = arr[i]; arr[i] = temp;
        permute(arr, l + 1, r); // Recurse
        // Backtrack: undo the swap
        temp = arr[l]; arr[l] = arr[i]; arr[i] = temp;
```

```
    }
}
// Time: O(n!) €"â there are n! permutations
```

4.4.4 N-Queens Problem (Conceptual)

Place N queens on an $N \times N$ chessboard so that no two queens attack each other.

Approach: Place queens column by column. For each column, try each row. If placing a queen is safe (no conflict with existing queens), place it and recurse. If no row works, backtrack.

```
// Check if placing queen at (row, col) is safe
int isSafe(int board[][20], int n, int row, int col) {
    for (int j = 0; j < col; j++)
        if (board[row][j]) return 0; // Same row
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j]) return 0; // Upper-left diagonal
    for (int i = row, j = col; i < n && j >= 0; i++, j--)
        if (board[i][j]) return 0; // Lower-left diagonal
    return 1;
}
```

Interview Tip

N-Queens is a classic interview question testing backtracking skills. The key is identifying the three constraints (row, column, diagonals) and implementing a clean `isSafe` check.

Practice Problems:

1. Solve a Sudoku puzzle using backtracking.
2. Find all paths from source to destination in a maze.
3. Generate all valid combinations of n pairs of parentheses.

4.5 Introduction to Dynamic Programming

4.5.1 What is Dynamic Programming?

Dynamic Programming (DP) solves problems by breaking them into **overlapping subproblems** and storing results to avoid recomputation.

Real-Life Analogy

Imagine you are asked: "What is $1+1+1+1+1+1$?" You count and say "6." Then someone asks: "What is $1+1+1+1+1+1+1$?" Instead of recounting, you say "7" because you *remember* the previous answer was 6 and just add 1. That is DP — remembering past results!

Two approaches:

- **Memoization (Top-Down):** Solve recursively but cache results.
- **Tabulation (Bottom-Up):** Build a table iteratively from smallest subproblems.

4.5.2 Fibonacci with DP

Without DP: $O(2^n)$ — exponentially slow.

With Memoization:

```c
int memo[100] = {0};
int fibMemo(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n]; // Already computed
    memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
    return memo[n];
}
// Time: O(n), Space: O(n)
```

With Tabulation:

```c
int fibTab(int n) {
    int dp[100];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
// Time: O(n), Space: O(n) €"â can be optimized to O(1)
```

4.5.3 Climbing Stairs

You can climb 1 or 2 stairs at a time. How many distinct ways to reach stair n?

```c
int climbStairs(int n) {
    if (n <= 2) return n;
```

```c
    int prev2 = 1, prev1 = 2, curr;
    for (int i = 3; i <= n; i++) {
        curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
// This is essentially Fibonacci! Time: O(n), Space: O(1)
```

4.5.4 0/1 Knapsack (Introductory Idea)

Given n items with weights and values, and a knapsack capacity W, maximize the total value without exceeding the capacity. Each item is either taken entirely or not (0/1 choice).

```c
int knapsack(int weights[], int values[], int n, int W) {
    int dp[n + 1][W + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (weights[i-1] <= w) {
                int include = values[i-1] + dp[i-1][w - weights[i-1]];
                int exclude = dp[i-1][w];
                dp[i][w] = include > exclude ? include : exclude;
            } else {
                dp[i][w] = dp[i-1][w]; // Can't include
            }
        }
    }
    return dp[n][W];
}
// Time: O(n * W), Space: O(n * W)
```

4.5.5 Longest Common Subsequence (Conceptual)

Find the longest subsequence common to two strings. For example, LCS of "ABCDE" and "ACE" is "ACE" (length 3).

```c
int lcs(char *X, char *Y, int m, int n) {
    int dp[m + 1][n + 1];
    for (int i = 0; i <= m; i++) {
```

```
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) dp[i][j] = 0;
            else if (X[i-1] == Y[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = dp[i-1][j] > dp[i][j-1] ?
                           dp[i-1][j] : dp[i][j-1];
        }
    }
    return dp[m][n];
}
// Time: O(m * n), Space: O(m * n)
```

Common Mistake

The most common DP mistake: confusing **subarray** (contiguous) with **subsequence** (not necessarily contiguous). Always clarify in an interview!

Interview Tip

DP problems follow a pattern: (1) Define the state, (2) Write the recurrence, (3) Identify base cases, (4) Decide top-down or bottom-up. Practice these four steps on every DP problem.

Practice Problems:

1. Minimum cost to reach the end of an array (min cost path).
2. Longest Increasing Subsequence.
3. Coin Change — minimum coins to make a given amount (DP version).

4.6 Sliding Window & Two Pointer Pattern

4.6.1 What is the Sliding Window?

The **sliding window** technique maintains a "window" (subarray) that slides across the array. Instead of recalculating from scratch, you **add the new element** entering the window and **remove the element** leaving it.

Real-Life Analogy

Imagine looking through a **window on a moving train**. As the train moves forward, you see new scenery entering the window and old scenery leaving. You do not need to look at the entire landscape again — just update what changed.

4.6.2 Fixed Window

Window size is constant (k). Example: Maximum sum of a subarray of size k.

```c
int maxSumSubarray(int arr[], int n, int k) {
    // Calculate sum of first window
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    int maxSum = windowSum;

    // Slide the window
    for (int i = k; i < n; i++) {
        windowSum += arr[i]; // Add new element
        windowSum -= arr[i - k]; // Remove old element
        if (windowSum > maxSum) maxSum = windowSum;
    }
    return maxSum;
}
// Time: O(n) €"â much better than O(n*k) brute force!
```

Dry Run with `arr = [2, 1, 5, 1, 3, 2]`, $k = 3$:

Window	Elements	Sum	Max	
[0..2]	2, 1, 5	8	8	
[1..3]	1, 5, 1	7	8	
[2..4]	5, 1, 3	9	9	
[3..5]	1, 3, 2	6	9	

4.6.3 Variable Window

Window size changes based on a condition. Example: Smallest subarray with sum $\geq$ target.

```c
int minSubarrayLen(int arr[], int n, int target) {
    int left = 0, sum = 0, minLen = n + 1;
    for (int right = 0; right < n; right++) {
        sum += arr[right]; // Expand window
        while (sum >= target) {
            int len = right - left + 1;
            if (len < minLen) minLen = len;
            sum -= arr[left]; // Shrink window
            left++;
        }
    }
    return (minLen == n + 1) ? 0 : minLen;
```

```
}
// Time: O(n) €"â each element is added and removed at most once
```

4.6.4 Subarray Problems

Longest Substring Without Repeating Characters:

```c
#include <string.h>
int longestUniqueSubstr(char str[]) {
    int lastSeen[256];
    memset(lastSeen, -1, sizeof(lastSeen));
    int maxLen = 0, left = 0;
    for (int right = 0; str[right] != '\0'; right++) {
        if (lastSeen[(int)str[right]] >= left) {
            left = lastSeen[(int)str[right]] + 1;
        }
        lastSeen[(int)str[right]] = right;
        int len = right - left + 1;
        if (len > maxLen) maxLen = len;
    }
    return maxLen;
}
// Time: O(n), Space: O(1) (fixed-size array)
```

Interview Tip

Sliding Window is one of the most asked patterns in interviews. The trick: identify if the problem asks about a *contiguous subarray/substring*. If yes, consider sliding window. Fixed window → known size. Variable window → expand right, shrink left.

Practice Problems:

1. Maximum of all subarrays of size k.
2. Count distinct elements in every window of size k.
3. Longest substring with at most K distinct characters.

Outcome

Student has developed strong algorithmic thinking and can apply searching, sorting, greedy, backtracking, dynamic programming, and sliding window strategies to solve complex computational problems efficiently in C.